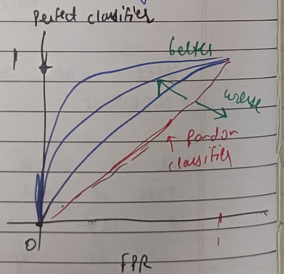
Receiver Operating Characteristics (ROC) curve

- True Positive Rate (TPR) = Recall = $TP / (TP + FN)$
- False Positive Rate (FPR) = $FP / (FP + TN)$

Axes of curve — X-axis: False Positive Rate
Y-axis: True Positive Rate

- How the curve is generated

- Say the classifier generates $P(y=1|x)$
- Set a threshold $\tau \in [0,1]$ s.t
  - any prediction $P(y=1|x) \geq \tau$ is labelled as class 1
  - any prediction $P(y=1|x) < \tau$ is labelled as class 0
- Start from $\tau = 0$ & increase it $\tau \in [0,1]$
- At each new threshold, recalculate TPR & FPR based on new predictions
- Plot each (FPR, TPR) as a point on graph to form a continuous curve



- Compute Area under Curve (AUC)
- A random binary classifier has TPR
  AUC - ROC = 0.5
- A perfect one has AUC - ROC = 1
- Similarly other plots - Precision vs Recall etc.

Understanding Model training and Evaluation

The challenge of performance evaluation:

Evaluating how well a machine learning model learns is inherently difficult because the entire population of possible data is never fully available.

- Practitioners must rely on a finite set of training data, which is typically smaller than desired
- A naive approach is to use all available data to train the model. However, this typically causes the model to overfit the training data
- Overfitting is especially pronounced with high-capacity models like neural networks. When this happens, the true error rate of the model is underestimated and it is not uncommon for an overfitted model to achieve 100% accuracy on training data while failing on new data

Data Terminology & Splitting

To avoid pitfalls of naive approach, datasets are split into three distinct categories under the strict assumption that all subsets are drawn from the exact same underlying data distribution

- Training data (Train set): This is the data subset used to actively train the model. The goal here is to find the best model parameters by maximizing the likelihood & minimizing loss over these specific samples.

- Development / Validation data / val set / :
  This subset is used strictly for tuning model "hyperparameters" & comparing diff models against one another. The models are never trained on this set. This can also be said to be a proxy for test set.

- Test Data: This is completely unseen data held out until very end to evaluate models performance.

Model fitting & Underfitting:

When fitting a model to data, several errors can arise if the model is too ~~simple~~ mathematically simple for underlying pattern in the dataset.
If a simple first-order curve (a straight line represented by the equation $y = w_0 + w_1 x$) is used to model data points that naturally follow a quadratic curve, the model will exhibit high errors on both train & val set.

- High errors are due to: (Sources)
  - Small no. of Training Samples!
  - Noise in training data!
  - Insufficient model capacity!
    - Too few parameters to capture model complexity.
    - eg line used to model points along a quadratic curve

This is the case of underfitting where the model is not capable of sufficiently expressing the training data (ie high training errors)

Bias & Underfitting

Bias refers to the error introduced by approximating a real world problem, which may be complex, with a model that is too simple.

- A model suffering with high bias is characterized by having both high train as well as high val / test error.
- This indicates that the model is underfitting the data, meaning it lacks the capacity to capture the underlying patterns.
- When model has high bias, providing it with more training data will not fix the issue, the val / test error remain high.

- eg if true f is quadratic & $\hat{f}$ is linear then it will have high bias
- An unbiased model is

$$E(\hat{f}) = f$$

Unbiased: $E[\hat{u}] = u$

Model fitting

Fifth order curve: $y = w_0 + w_1 x + w_2 x^2 + w_3 x^3 + w_4 x^4 + w_5 x^5$
Is the above a linear model?
Short ans: Yes it is a linear model
Long Answer:
The question should be is the function $y = f(x, w)$ linear in w? Yes since all w_i terms have power 1

Linearity

A function $g(u)$ is linear if it satisfies both of the following properties:

- Superposition: $g(u+v) = g(u) + g(v)$
- Homogeneity: $g(cu) = cg(u)$

- Our linear model $y = f(x, w) = wx$ is linear in x and w

- What about $y = f_1(x, w_0, w_1) = w_0 + w_1 x$

If $f_1(x, w_0, w_1)$ linear in x?

No! does not satisfy $g(cu) = cg(u)$
It is an Affine function!!
linear in w! yes ✓

- What about $y = f_2(x, w) = w_0 + w_1 x + w_2 x^2 + \dots + w_5 x^5$?

If $f_2(x, w)$ linear x? → No!
Is it linear in w? → yes ✓

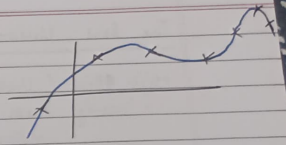
Model fitting

If we take a 5th order curve
$y = w_0 + w_1 x_0 + w_2 x^1 + \dots + w_5 x^5$

A try to fit to a first true f is quadratic

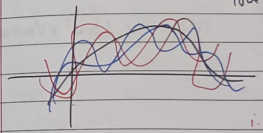
Train

- Model fits perfectly to data
- Zero train errors



but it overfits
if we get less error on val / test than we just got lucky.

If we do k iterations of train on some data we get



$\hat{f}$'s vary all $\hat{f}$ are very diff from each other
i.e model fits the noise it overfits

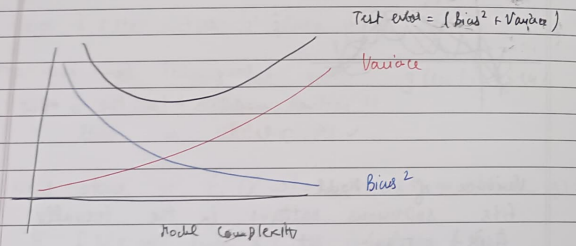
Variance of a Model:
fits spurious patterns in the (usually small, finite) train set.
- characterized by low train error, but high val / test error.

Indicates model is overfitting the train set
- Typically of high capacity models, i.e models with many parameters

Mitigation approaches
- Use a larger train set
- Regularization
- Feature Engineering.

The Bias Variance Trade off:

- High Bias / Underfitting: Model complexity too low
  - Training error & Test / Val error are both too high
- High Variance / Overfitting: Model complexity is too high
  - Train error is low, but test / val error too high
- Correct fit: Model complexity is just right
  - Both train & Test / Val errors are low enough
  - In our eg it is achieved by fitting a quadratic curve



How does Regularization help:

Fifth order curve

$y = w_0 + w_1 x + w_2 x^2 + w_3 x^3 + w_4 x^4 + w_5 x^5$

what if

$w_3 = w_4 = w_5 = 0$

How do we achieve it?

Let $\vec{w} = [w_0 \; \ldots \; w_5]^T$

& $\vec{x} = [1 \; x_1 \; \ldots \; x_5]$

then the learned model has parameters

$$\vec{w}^* = \arg\min_{\vec{w}} \sum_{i=1}^{N} \left[y^{(i)} - \vec{w}^T x^{(i)} \right]^2$$

$$\text{s.t} \quad \|\vec{w}_3\| \leq 3$$

Constraints on model params

L_0 norm is no. of non zero elements in a vector

now by putting this constrain it is most likely the case w_3, w_4, w_5 will go to zero after training

$$\vec{w}^* = \arg\min_{\vec{w}} \sum_{i=1}^{N} \left(y^{(i)} - \vec{w}^T x^{(i)} \right)^2 + \lambda \|\vec{w}\|_0$$

(lambda times L_0 norm)

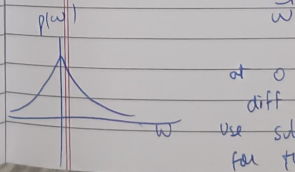
Regularization term (not differentiable)

Since it is not diff

a relaxed problem is used a L_1 norm

$$\vec{w}^* = \arg\min_{\vec{w}} \sum_{i=1}^{N} \left(y^{(i)} - w^T x^{(i)} \right)^2 + \lambda \|\vec{w}\|_1$$

Sparsity promoting Regularization term (also differentiable & convex)



at 0 not diff we use subgradients for that

K fold cross validation :

as K ↑ Val set ↓ Train set in diff runs become more correlated so cv error does not reduce much variance also as Val set dec so this inc the variability of accuracy

e.g. if Val set = 1 so acc either 100% or 0%

K ↓ large val set ⇒ generally more stable cross validation estimate ⇒ lower variance

Linear Regression

$y \sim N(f(x^i, w), \sigma^2)$

$L(y, f(x,w)) = \sum L(f(x^i, w)) + \lambda \|w\|$

Regularization

- Gaussian prior on L_2 or Ridge Regression
  (helps in reducing variance)
- Laplacian prior on L_1 regularization or LASSO (Least absolute shrinkage & selection operator)
  $R(w) = \|w\|_1$ (Promotes sparsity)
  also more case for Laplacian -- continue ---
- Gaussian + Laplacian Mixture Prior on Elastic net

$R(w) = \|w_0\|_2^2 + \alpha_1 \|w\|_1$

Bayes thing

$L(w) = -l(w) + \lambda R(w)$

data fidelity ↑

Logistic Regression

$$P(y|x, p) = \begin{cases} p & , y = 1 \\ 1-p & , y = 0 \end{cases}$$

Used logit function (log of odds)

loss function

$L(w) = -l(w) + \lambda R(w)$

happens to be binary cross entropy loss for ($\lambda = 0$)

So we can also apply regularization to classification things..

Probabilistic Classifier : Classification Rule :

Classification is then performed as:

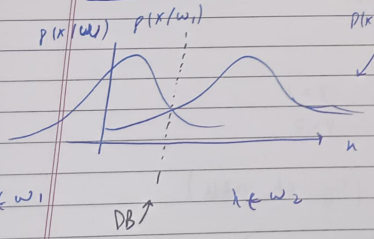
If $P(\hat{y}^i = 1 \mid x^i, w) > P(\hat{y}^i = 0 \mid x^i, w)$
the label

$\hat{y}^i = 1$ else $\hat{y}^i = 0$

for M class

$$\hat{y}^i = \underset{y \in \{0, \ldots, M\}}{\arg\max}\ P(f(x^i, w) = y \mid x^i, w)$$

Decision boundary



Discriminative vs Generative Probabilistic Classifiers :

For Binary Classification, we are interested in modeling $y \in \{0, 1\}$

- Interested in: $P(y \mid x)$

Discriminative classifier = directly models for $P(y \mid x)$
the output is $P(y \mid x)$
eg logistic regression

$$P(y = 1 \mid x, w) = \frac{1}{1 + e^{-w^T x}}$$

A generative classifier leverages the data distribution, given a class.

Applying Bayes

$$P(y \mid x) = \frac{P(x \mid y)\, P(y)}{P(x)}$$

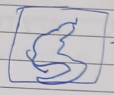
$P(x)$ can be our distribution it does not depend on y so for classification rule we can ignore this $P(x)$, also it will same for ~~both~~ all classes etc.. ...
So for classification we can ignore $P(x)$ also estimating it is highly non trivial.
$P(y)$ is the prior prob of a class

$P(x \mid y)$ - given a particular class what is the prob distribution of X

we can sample new data from $P(x|y)$ if we know the parameters and distribution of it. We can generate new data once we model this. That's where the generative term comes from.

Eg Digit recognition

pixels 30×30



⇒ (Classifier) → 5

- $\{x_1, \ldots x_n\} \in \{0,1\}$
- $Y \in \{5, 6\}$ (predict whether it is 5 or 6)

a good strategy is

$$\underset{Y}{\text{argmax}}\ P(Y|X_1 \ldots X_n)$$

How can we figure this out...

For eg If we want to figure out $P(Y|X)$ for coin flip we would probably flip it N times & approximate

Now for this we need to observe all possible vals of X_i's if 2^n possible combinations So estimating it is very hard.

Using bayes rule

$$\text{Posterior} \rightarrow P(Y|X_1 \ldots X_n) = \frac{P(X_1 \ldots X_n|Y)\,P(Y)}{P(X_1, \ldots X_n)}$$

(likelihood, prior)

finding likelihood is much easier given Y (digit) & all val of pixels & prior is from prev knowledge of class.

So given a category we can collect input data & estimate the distribution.

$$P(Y=5|X_1 \ldots X_n) = \frac{P(X_1 \ldots X_n|Y=5)\,P(Y=5)}{P(X_1 \ldots X_n|Y=5)\,P(Y=5) + P(X_1 \ldots X_n|Y=6)\,P(Y=6)}$$

$$P(Y=6|X_1 \ldots X_n) = \frac{P(X_1 \ldots X_n|Y=6)\,P(Y=6)}{P(X_1 \ldots X_n|Y=5)\,P(Y=5) + \ldots}$$

The joint distribution of (X_1, X_2, X_3, X_4) given $Y=1$ is given by assigning probabilities to each of the possible inputs

X_1	X_2	X_3	X_4	$P(X_1, X_2, X_3, X_4 \mid Y=1)$
0	0	0	0	p_1
0	0	0	1	
0	0	1	0	
.	.	.	.	
1	1	1	1	p_{15}

Total # parameters needed to define the likelihood $P(X_1, X_2, X_3, X_4 \mid Y=1)$

$= 15$

$= 2^4 - 1$

$= 2^n - 1$

$1 - \sum p_i$

For a 30×30 image, pixels are $X_1, X_2, \ldots X_n \mid n = 900$

$\Rightarrow$ # parameters $2^{900} - 1$

The Naive Bayes Model

Naive bayes assumption:

Assume that all features are independent given the class label Y

Mathematically

$$P(X_1, \ldots X_n \mid Y) = \prod_{i=1}^{n} P(X_i \mid Y)$$

- # of parameters for modelling $P(X_1 \mid Y) \ldots P(X_n \mid Y)$
  $= 2n$

Actually this is a very simplified model we can also group into factors like we can say one row is dependent & rows are independent of each other so it will be if each row has m pixels

$$= \prod_{i=1}^{} P(X_i \ldots X_{i+m} \mid Y)$$

this is thing used by bayesian networks.

Naive bayes also works pretty well and gives a good baseline.

classmate
Date
Page

assume $P(x|y=0) = N(\mu_0, \Sigma)$

$P(x|y=1) = N(\mu_0, \Sigma)$

$y \sim$ Bernoulli (ϕ)

$P(x) = \frac{1}{(2\pi)^{p/2}|\Sigma|^{1/2}} e^{-\frac{1}{2}(x-\mu_0)^T \Sigma^{-1}(x-\mu_0)}$

assume $P(y=1) = P(y=0)$

$= \frac{1}{(2\pi)^{p/2}|\Sigma|^{1/2}} e^{-\frac{1}{2}(x-\mu_1)^T \Sigma^{-1}(x-\mu_1)}$

$e^{-\frac{1}{2}(x-\mu_0)^T \Sigma^{-1}(x-\mu_0) - \frac{1}{2}(x-\mu_1)^T \Sigma^{-1}(x-\mu_1)}$

$(\mu_0 - \mu_1)^T \Sigma^{-1} (x - \mu_1) = (\mu_0 - \mu_1)^T \Sigma^{-1} (x - \mu_0)$

$(\mu_0 - \mu_1)^T \Sigma^{-1} \left(x - \frac{\mu_1 + \mu_0}{2} \right) = (\mu_0 - \mu_1)^T \Sigma^{-1} x - (\mu_0 - \mu_1)^T \Sigma^{-1} \frac{\mu_1 + \mu_0}{2}$

$\mu_0^T \Sigma^{-1} \mu_0 - \mu_1^T \Sigma^{-1} \mu_0 + \mu_0^T \Sigma^{-1} \mu_1 - \mu_1^T \Sigma^{-1} \mu_1$

$\mu_0^T \Sigma^{-1} \mu_0 - \mu_1^T \Sigma^{-1} \mu_1$

$(\mu_0 - \mu_1)^T \Sigma^{-1} x = \frac{1}{2} (\mu_0 - \mu_1)^T \Sigma^{-1} (\mu_0 + \mu_1)$

$(\mu_0 - \mu_1)^T \Sigma^{-1} x = \frac{1}{2} (\mu_0 - \mu_1)^T \Sigma^{-1} (\mu_0 + \mu_1)$

$(\mu_0 - \mu_1)^T \Sigma^{-1} x + \frac{1}{2} (\mu_0 - \mu_1)^T \Sigma^{-1} (\mu_0 + \mu_1) = 0$

$\theta^T x + b = 0$

hyperplane

$\theta = (\mu_0 - \mu_1)^T \Sigma^{-1}$

$b = \frac{1}{2} (\mu_0 - \mu_1)^T \Sigma^{-1} (\mu_0 + \mu_1)$

$D = \{(x^i, y^i)\}_{i=1}^{N}$

$l(\cdot) = \prod_{i=1}^{N} p(y^i, x^i; \phi, \mu_0, \mu_1, \Sigma)$

$l(\phi, \mu_0, \mu_1, \Sigma) = \prod_{i=1}^{N} p(x^i | y^i) \, p(y^i)$

taking log

$= \sum_{i=1}^{N} \log\left(p(x^i | y^i) + \log(y^{(i)}\right)$

$= \sum_{i=1}^{N} I(y^i = 0) \left(\log\left[p(x^{(i)} | y^{(i)} = 0) + \log(\phi_0^{(i)})\right)\right.$

$+ I(y^{(i)} = 1) \left(\log\left(p(x^{(i)} | y^{(i)} = 1)\right) + \log(\phi_1^{(i)})\right)$

$+ I(y^{(i)} = 2) \left(\log\left(p(x^{(i)} | y^{(i)} = 2)\right) + \log(\phi_2^{(i)})\right)$

$+ \cdots$

$+ I(y^{(i)} = K) \left(\log\left(p(x^{(i)} | y^{(i)} = K)\right) + \log(\phi_K^{(i)})\right)$

$\{x_1, \ldots, x_n\}$

y_i is not known

We define z as another var

$z \in \{0, 1\}$

$p(x, z; \mu_0, \mu_1, \Sigma_0, \Sigma_1)\ p(z; \phi)$

Bias - error what model generates when trained on ∞ amount of data

$E(\bar{x}) = \frac{1}{N} \sum_{i=1}^{N} E(x^i)$

$= \frac{1}{N} N\mu$

$= \mu$

$Var(\bar{x}) = \frac{1}{N^2} \sum Var(x^i)$

$= \frac{1}{N} \sigma^2$

Training error is an estimate of bias as we have already seen the data & we have the best fit which can model the data

$E(f - \hat{f})$

Variance - estimate of performance on unseen data

Multiclass Naive Bias derivation

$\{(x^1, y^1), \ldots (x^m, y^m)\}$

labels (y) : Multiclass where $y^{(i)} \in \{1, \ldots k\}$

$x^{(i)} = (x_1^{(i)} \ldots x_n^{(i)})^T$

$x_j \in \{1, 2, \ldots V_j\}$

$\phi_{y=k} = P(y = k)$; $\sum_{k=1}^{k} \phi_k = 1$

$\phi_{j=v|y=k} = P(x_j = v \mid y = k)$; $\sum_{v=1}^{V_j} \phi_{j=v|y=k} = 1$

$P(x, y; \phi_{y=k}, \phi_{j=v|y=1}, \phi_{j=v,y=2} \ldots)$

$P(x, y) = P(x \mid y) P(y)$

$P(x^{(i)}, y^{(i)}) = P(x^i \mid y^i) P(y^i)$ ← naive bias ass

$= \left(\prod_{j=1}^{n} P(x_j^i \mid y^i)\right) P(y^i)$

$L(\theta) = \prod_{i=1}^{m} \left[\left(\prod_{j=1}^{n} P(x_j^i \mid y^i)\right) P(y^i)\right]$

log likelihood

$\ell(\theta) = \log(L(\theta)) = \sum_{i=1}^{m} \left(\log(P(y^i)) + \sum_{j=1}^{n} \log(P(x_j^i \mid y^i)) \right)$

we can write

$$P(y^{(i)}) = \prod_{k=1}^{k} (\phi_{y=k})^{I\{y^{(i)}=k\}}$$

$$P(x_j^i / y^{(i)}=k) = \prod_{v=1}^{v_i} (\phi_{j=v|y=k})^{I\{x_j^i=v,\ y^i=k\}}$$

So Taking log

$$\log(P(y^{(i)})) = \sum_{k=1}^{k} I\{y^{(i)}=k\} \log(\phi_{y=k})$$

$$\log\left(P(x_j^i / y^i=k)\right) = \sum_{v=1}^{v_i} I\{x_j^i=v, y^i=k\} \log(\phi_{j=v|y=k})$$

So

$$\ell(\theta) = \sum_{i=1}^{m} \left[\sum_{k=1}^{k} I\{y^i=k\} \log(\phi_{y=k}) + \sum_{j=1}^{n} \sum_{k=1}^{k} \sum_{v=1}^{v_j} I\{x_j^i=v, y^i=k\} \log(\phi_{j=v|y=k}) \right]$$

Now we will maximize it will lagrange

Deriving $\phi_{y=k}$

$$L = \ell(\theta) + \lambda\left(1 - \sum_{k=1}^{k} \phi_{k=k}\right)$$

$$\frac{\partial L}{\partial \phi_{y=k}} = \sum_{i=1}^{m} \frac{I\{y^i=k\}}{\phi_{y=k}} - \lambda = 0$$

$$\phi_{y=k} = \frac{\sum_{i=1}^{m} I\{y^{(i)}=k\}}{\lambda}$$

To find λ

$$\sum_{k=1}^{k} \phi_{y=k} = 1 \quad - \frac{1}{\lambda} \sum_{k} \sum_{i} I\{y^{(i)}=k\} =$$

$$= \frac{1}{\lambda} m = 1$$

$$\lambda = m$$

$$\phi_{y=k} = \frac{\sum_{i=1}^{m} I\{y^{(i)}=k\}}{m}$$

deriving $\phi_{j=v|y=k}$

$$L = \sum_{i} \sum_{j} \sum_{k} \sum_{\ell} I\{x_j^i=\ell, y^i=k\} \log(\phi_{j=v|y=k}) + \lambda_{jk}\left(1 - \sum_{\ell} \phi_{j=\ell|y=k}\right)$$

$$\frac{\partial L}{\partial \phi_{j=\ell|y=k}} = \sum_{i} \frac{I\{\quad\}}{\phi_{j=\ell|y=k}} - \lambda_{jk} = 0$$

$$L = \ell(\theta) + \sum_{j=1}^{n} \sum_{k=1}^{k} \lambda_{jk}\left(1 - \sum_{\ell=1}^{v_j} \phi_{j=\ell|y=k}\right)$$

Here this is like $\sum_{l=1}^{3} \phi_{2=l/y=1} = \phi_{2=1/y=1} + \phi_{2=2/y=1}$ (assuming) $\phi_{2=3/y=1}$

feature no. / Val of that feature

$$\frac{\partial L}{\partial \phi_{j=k/y=k}} = \frac{\sum_{i=1}^{m} I\{x_j^{(i)}=l, y^{(i)}=k\}}{\phi_{j=k/y=k}} - \lambda_{jk} = 0$$

$$\phi_{j=k/y=k} = \frac{\sum_{i=1}^{m} I\{x_j^{(i)}=l ; y^{(i)}=k\}}{\lambda_{jk}}$$

Find λ_{jk}

$$\sum_{l=1}^{V_j} \phi_{j=l/y=k} = \frac{1}{\lambda_{jk}} \{\{ I \{ \quad \}$$

$$1 = \frac{1}{\lambda_{jk}} \sum_{i=1}^{m} I\{y^{(i)}=k\}$$

$$\lambda_{jk} = \sum_{i=1}^{m} I(y^{(i)}=k\}$$

$$\phi_{j=k/y=k} = \frac{\sum_{i=1}^{m} I(x_j^{(i)}=l, y^{(i)}=k\}}{\sum_{i=1}^{m} I\{y^{(i)}=k\}}$$